AGENTIC DEVELOPMENT · 2026

The model is the engine. The harness is the car.

What makes a good agentic coding harness, and why it matters more than the model.

SECTION 01 / 05

The Problem

Agents can't remember between sessions, and every obvious fix for it fails.

01

2025 was the “Year of the Agent.” It didn't land.

The hype was model-led. In production the model alone fell short, expensively, and the studies agree on the reason: it wasn't the model.

95%

of enterprise GenAI pilots showed no measurable P&L impact

MIT NANDA, 300+ deployments: the divide “is not driven by model quality” but by approach. The dividing line is memory, workflow integration, and learning.

80%

of AI projects fail, twice the rate of non-AI software

RAND, from 65 practitioner interviews: of five root causes, only one touches model capability. The rest are problem-alignment, data, and integration.

Across MIT and RAND, one verdict: the missing variable wasn't intelligence, it was the **harness around the model**.

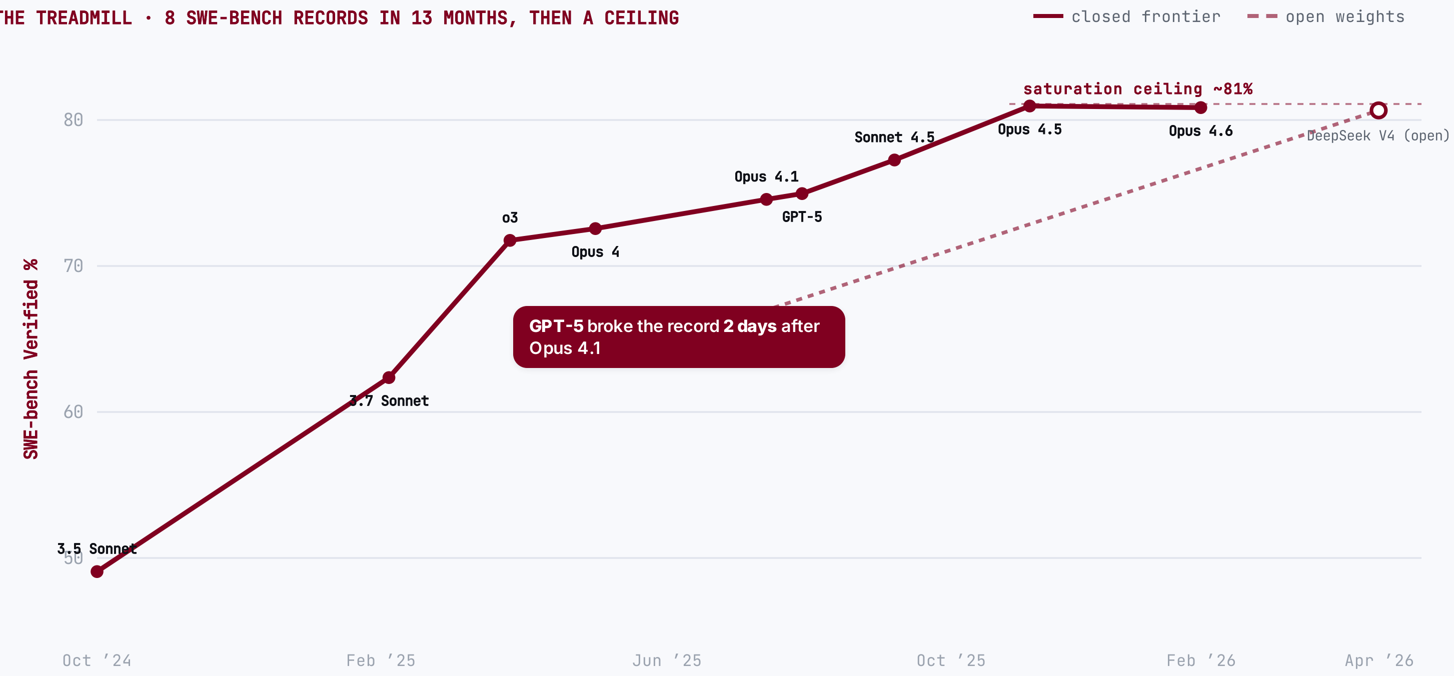
MIT Project NANDA · The GenAI Divide: State of AI in Business 2025 (95% of pilots: no measurable P&L impact) · via Fortune (2025) — fortune.com | RAND Corporation · The Root Caus…

THE PROBLEM · THE MODEL IS NOT THE MOAT

Models leapfrog every benchmark, then commoditize.

A new frontier model tops every benchmark… until the next does, **days** later. Then it saturates ~81%, open weights reach **parity**, and the benchmark itself gets retired.

THE TREADMILL · 8 SWE-BENCH RECORDS IN 13 MONTHS, THEN A CEILING



Open weights, now at parity

≈ 80.6% = the frontier

DeepSeek V4-Pro matches the closed SOTA on SWE-bench Verified (Apr '26). Composite gap now ~**4 months / 8 ECI pts** (Epoch AI, May '26).

The benchmark itself broke

59% of hard tests flawed

OpenAI **retired SWE-bench Verified** (Feb '26): frontier models had trained on the answers; it's saturated. → SWE-bench Pro.

Inference is collapsing

≈ 280× cheaper / 18 mo

GPT-3.5-level tokens: \$20.00 → \$0.07 per M. ~10× every year: "LLMflation."

"The models are getting commoditized."

Satya Nadella · Microsoft CEO · BG2 Pod, Dec 2024

The model is a commodity, not a moat.

Whatever tops the leaderboard this week is matched, undercut and open-weighted within months, and the benchmark itself gets retired.

SECTION 02 / 05

The Agentic Harness

The pilots that stalled tuned the model. The ones that shipped engineered the harness.

02

Hold the model fixed, change the harness.

Three current frontier models, 2025 to 2026. In each, the model weights never change; only the harness around them does.

TERMINAL-BENCH 2 · CLAUDE OPUS 4.6

Swap only the agent harness: Terminus 2 → the Meta-Harness.

62.9% → **76.4%**

+13.5 points

share of tasks resolved (0 to 100%)

Terminal-Bench 2 leaderboard · tbench.ai · 2026

SWE-BENCH VERIFIED · CLAUDE OPUS 4

Add a control layer: sample in parallel, discard patches that fail the tests, rerank.

72.5% → **79.4%**

+6.9 points

share of tasks resolved (0 to 100%)

Anthropic · Introducing Claude 4 · 2025

TERMINAL-BENCH 2 · GPT-5.4

Auto-evolve the harness over 10 iterations; it beats the human-built Codex-CLI.

69.7% → **77.0%**

+7.3 points

share of tasks resolved (0 to 100%)

AHE · arXiv 2604.25850 · 2026 preprint

Same weights every time. The harness is the differentiator, ***not the model****.*

Performance lives in the tooling around the model, not the model.

The effort runs one way; the leverage runs the other. Teams tune the prompt, but performance comes from the tooling, control, and memory wrapped around the model.

WHERE TEAMS LOOK

The model
+ the prompt

swapped, tuned, and re-prompted

table stakes once it is capable

THE INVERSION

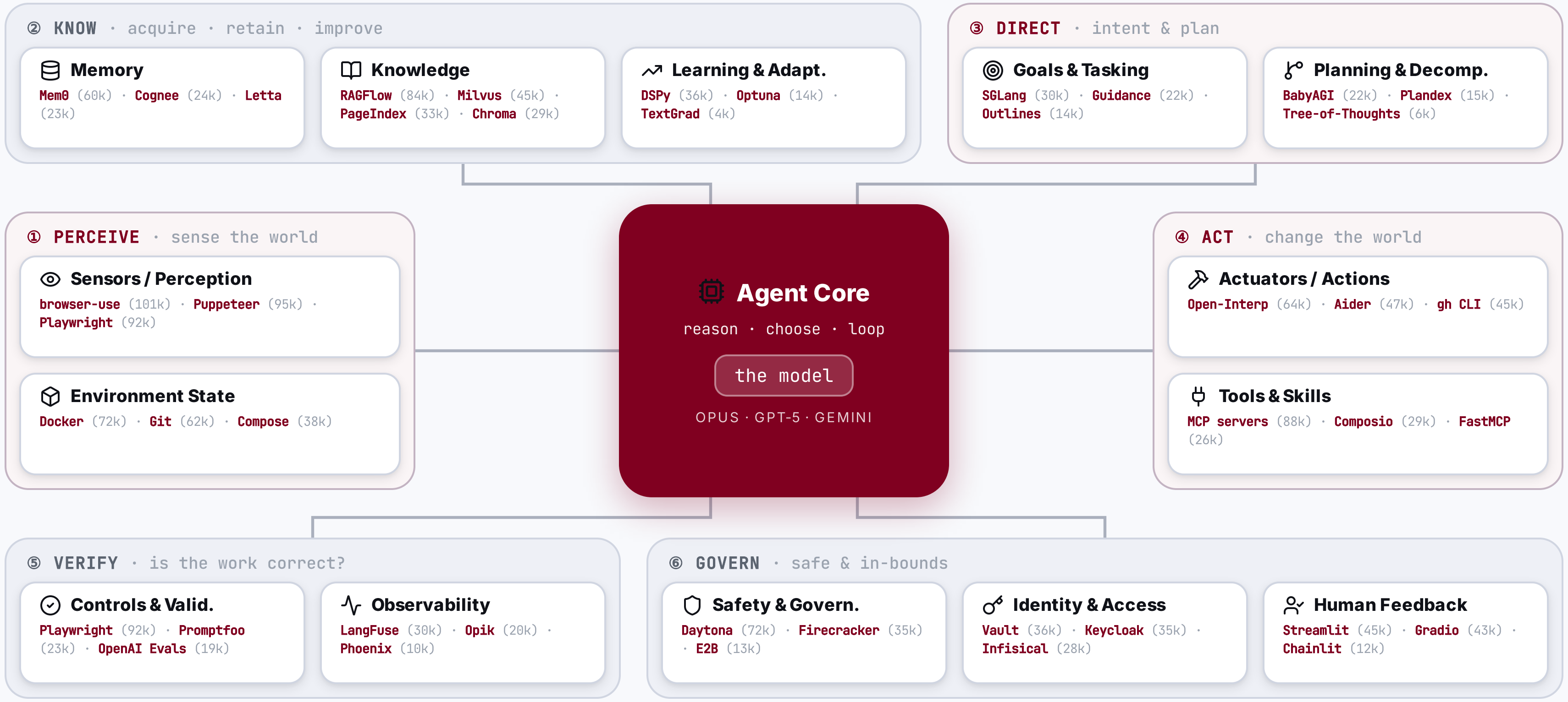
WHERE PERFORMANCE ACTUALLY LIVES **THE HARNESS**

Tools
read · run · search · sandbox

Control layer
context · execution · recovery

Long-term memory
files that persist across sessions

The agent core, wrapped in *14 capabilities*, and who leads each.



FRAMEWORKS SPAN MANY OF THESE

LangChain 140k · MetaGPT 69k · AutoGen 59k · CrewAI 55k · LlamaIndex 51k · LangGraph 36k · OpenAI Agents SDK 27k

Do you really think software engineering is dead?

That map was **50 tools** wrapping a single model: each its own system, all to be wired together and kept alive.

50 **tools around one model**

and only the top few per capability; 80+ ship in the wild

Each tool is its own system
its own API, config, failure modes and learning curve

Orchestration is the hard part
wiring 50 moving parts into one reliable, debuggable loop

Maintenance never stops
versions drift, tools break, the harness needs constant upkeep

THE LADDER OF ABSTRACTION · every rung was once "the end of programming"

The agentic harness	2025 · you are here
Cloud · serverless · DevOps	2010s
Libraries & frameworks	1990s–2000s
Compilers & high-level languages	1950s–70s
Assembly	1950s
Machine code · punch cards	1940s

No. We've just climbed another rung on the ladder of abstraction.
Software engineering didn't disappear; building and maintaining this harness **is** the engineering now.

Internal research · Agentic Harness: Capabilities map (15 functional domains, 80+ real shipping tools) (2026)

BUILDING WITH AGENTS · IN PRACTICE

Dev Tooling

The editor, model, agent-CLI and IDE you build with, open-source or proprietary, and how engineers climb from vibe-coding to real agentic engineering. The harness and the method underneath stay the same.

Open-source or proprietary: **your setup, your call.**

Three tiers: bring a model, compose a custom harness, or run a full IDE, each available **fully open** or **fully proprietary**. What doesn't change is the method underneath.

	OPEN-SOURCE · self-hostable	**PROPRIETARY** · hosted
Full IDE ALL-IN-ONE	VS Code ★187k · Zed ★86k · Continue ★35k · Tabby ★34k Zed standalone · VS Code + ext	Cursor · Windsurf · Antigravity · Kiro closed-source · bundled
Custom harness EDITOR + AGENT	opencode ★180k · Cline ★64k · Goose ★50k · Aider ★47k · Crush ★26k open + model-agnostic	Claude Code ★135k · Codex ★94k · Copilot · Antigravity CLI · Amp proprietary · vendor-hosted
Inference & routing OPTIONAL	Ollama ★175k · llama.cpp ★118k · vLLM ★85k · LiteLLM ★52k self-host · open	OpenRouter · Groq · Hugging Face · Bedrock · Vertex AI · Azure OpenAI managed · cloud gateway
Bring your own model THE MODEL	DeepSeek · Llama · Qwen · gpt-oss · Mistral open weights · self-hostable	Claude · GPT-5 · Gemini 3 · Grok frontier · hosted

The method
THE CONSTANT, EITHER SIDE

`spec` → `plan` → `grill-me` → `peer-review` → `loop` · artefacts · rules · skills · the **metamodel**

From “vibe coding” to *agentic engineering*: five levels.

Autonomy climbs and the harness thickens, but the human moves **up** the stack (prompter → orchestrator), never out of it.

WHAT CHANGES PER L	1 Ad-hoc · “vibe coding”	2 Assisted	3 Spec-driven + harness	4 Full harness · loop	5 Multi-agent fleet
PROCESS	Chat prompts; accept what runs	Deliberate prompts; review each diff	Spec/PRD → agent runs build/test loop	Agent owns implement→test→commit	Orchestrator → parallel agents → PRs
TOOLING	none	CLAUDE.md / AGENTS.md	docs/ · rules · skills	+ hooks · MCP · fix_plan · tests	+ git worktree per agent
HARNESS	raw model	thin: one context file	guides + first sensors (hooks)	complete + loop driver (ralph.sh)	fleet: subagents · worktrees · critic
HUMAN	**Prompter** every turn · shallow — Karpathy **vibe coding**	**Author** reviews every line — Cursor **Copilot**	**Director** approves at gates — Spec Kit · Kiro **clew**	**Supervisor** goals + escalations — **Ralph loop** Huntley	**Orchestrator** writes specs · tunes — Anthropic **+90%**

INCREASING AUTONOMY · DEEPER HARNESS →

role rises with every step ↗, never leaves the loop

⚠ **Even L5 is *supervised* autonomy, not hands-off.** METR: experienced devs were 19% *slower* on mature codebases; SWE-bench Pro tops out ~23%; Ralph’s author: “no way I’d use it on an existing codebase.” The guardrail against Thoughtworks’ **“cognitive debt”** is keeping humans on standards, architecture & final sign-off.

THE HARDEST LAYER

Memory

Bolt-on memory is now commodity: Mem0, Letta, vector search. What no tool ships is memory that holds its structure and traceability across refactors, and that is what the harness has to build.

An agent has four memories. One is volatile, three persist.

The **CoALA** framework (Princeton, TMLR 2024) names them. Everything the agent knows is loaded through the one volatile window to be used.

WORKING MEMORY · VOLATILE

The context window

the active state the model is given on every LLM call, rebuilt each turn

↘ Context rot

PERSISTENT · SURVIVES ACROSS SESSIONS · LOADED INTO THE WINDOW ON DEMAND

SEMANTIC MEMORY

AGENTS.md · CLAUDE.md

knowledge of the world and itself

↘ The one big file

PROCEDURAL MEMORY

Skills + progressive disclosure

the code and procedures for how it acts

↘ Discovery & staleness

EPISODIC MEMORY

Past sessions & transcripts

experience from earlier decision cycles

↘ Keep vs forget

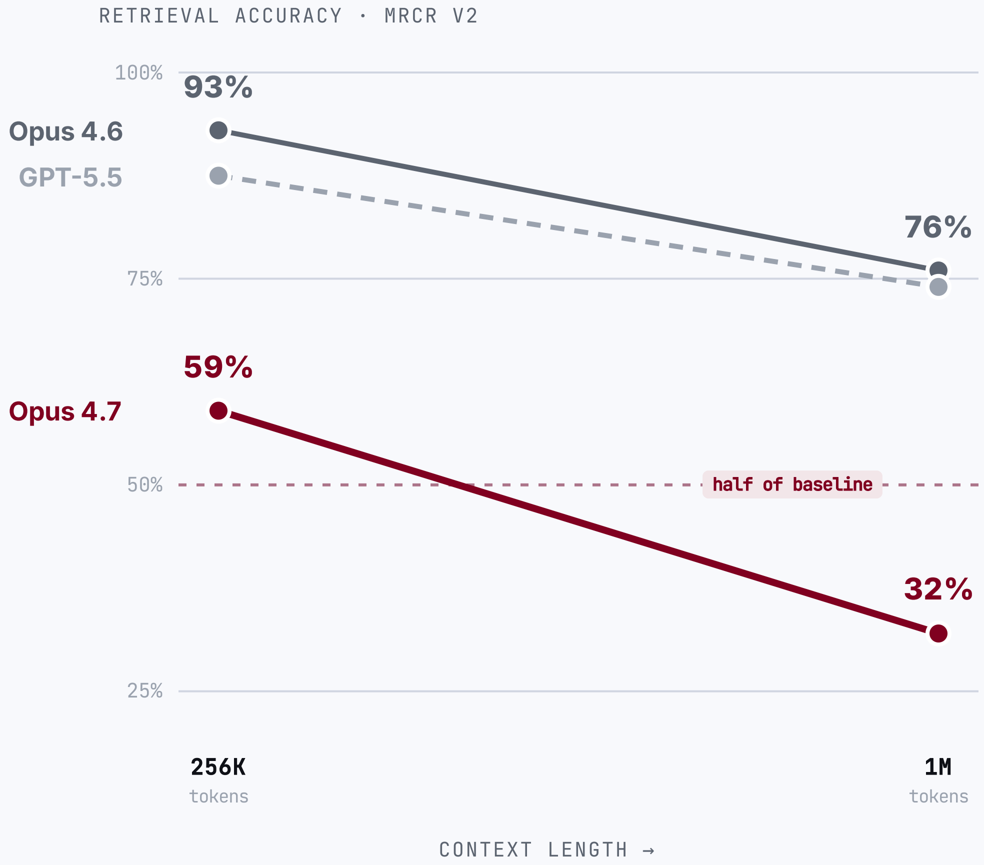
One volatile window, three persistent stores. Each fails in its own way, next four slides.

Sumers, Yao, Narasimhan & Griffiths (Princeton) · Cognitive Architectures for Language Agents (CoALA): four memory modules (working, episodic, semantic, procedural) · TMLR 2024 (20…

No. A bigger context window is not bigger memory.

Working memory is the live context window: the only state the model actually sees, and it degrades as you fill it.

WHERE IT BREAKS



The advertised window is **not** the usable window.

59% → 32%

Claude Opus 4.7 loses ~half its multi-needle retrieval from 256K to 1M tokens. Opus 4.6 holds better (93% → 76%).

Codex too

the GPT-5.x line behind Codex CLI drops ~87% → 74% at long context, a proxy; no per-length figure is published for the Codex-tuned model.

11 of 13

models fell below half their accuracy by 32K tokens. Context rot isn't new (NoLiMa, 2025).

→ **The fix:** curate, don't cram. Retrieve only the few high-signal tokens the task needs into a fresh window; treat context as a budget, not a bucket.

Writing it down isn't memory either.

Semantic memory is what the agent knows about your project: the `AGENTS.md` and `CLAUDE.md` it reads in at the start of every session.

WHERE IT BREAKS

OpenAI shipped **1M lines**, **0 hand-written**. Then one big `AGENTS.md` broke.

AGENTS.md — ONE BIG FILE

+ 900 more rules · no one can tell which still apply

- **Crowds out the task**
- **Too much guidance, none**
- **Rots silently**
- **Can't be verified**

→ **The fix:** a ~100-line table of contents pointing into a structured `docs/` tree. Memory has to be a **checkable map**, not a flat file you hope holds.

Skills tame context bloat, but only if the agent picks the right one.

Procedural memory is the agent's reusable know-how: skills, surfaced by progressive disclosure. Names sit in context; the full instructions load only when judged relevant.

WHERE IT BREAKS

Discovery rides on one sentence. Claude chooses from 100+ skills using only each description. A vague one never fires.

The metadata is never free. Every skill's description permanently shares the context budget with your task.

It rots silently. Bundled instructions drift; "avoid time-sensitive information" is the only guardrail.

The eager alternative is worse. Codex `AGENTS.md` and Aider conventions load every turn, no relevance gate.

→ **The fix:** a curated, sharply-described, version-controlled skill set. The procedure layer rewards **structure**, not accumulation.

The hard part isn't remembering. It's forgetting.

Episodic memory is experience from past sessions: the transcripts of what the agent did, and what they taught.

WHERE IT BREAKS

Why "remember everything" fails

>90%
of the token cost is wasted re-stuffing full history each turn, vs. a managed memory store.

- ✕ A fixed window can't hold months of sessions, so old context is silently dropped.
- ✕ Recall gets noisy: the more you retain, the harder the right memory is to find.
- ✕ Stale memories don't just bloat, they actively mislead the agent.

HOW THE FIELD MANAGES IT

Score & decay
Rate each memory's importance, decay it by recency, rank by relevance.
`Generative Agents`

Extract & consolidate
Keep only the salient facts, merge them, retrieve a few. `Mem0`

The open frontier
What to evict, "learned forgetting," is still listed **unsolved** in 2026. `Memory survey`

→ **The fix:** treat retention as a decision, not a default. Choosing what's worth keeping needs a **model of what matters**, which is exactly what no off-the-shelf store has.

The memory tools are real. They stop at recall.

A whole ecosystem now gives agents memory, even relational memory. Each thing it ships has a precise counterpart it **doesn't**.

✓ MEMORY YOU CAN BUY TODAY

Flat recall
Mem0 · LangMem · vector DBs
Facts and past chats, retrieved by similarity.

Relational memory
Zep / Graphiti · Cognee
Knowledge graphs of entities and relationships.

Coding-agent memory
Cursor · Claude Code · Aider
Rules files, auto-noted facts, code maps.

✕ MEMORY YOU STILL CAN'T

A model of what must be true
The product itself, not snippets about it.

Traceability across refactors
Links that stay valid when you restructure.

Referential integrity, enforced
Checked mechanically, not prose that rots.

Everything on the left you can assemble. The right is what the harness still has to build, and cross-session coherence is, per the 2026 survey, ***"largely unsolved."***

Databricks AI Research · Memory scaling for AI agents (as models converge, memory is the differentiator) (2026) — databricks.com | arXiv 2603.07670 · Memory for Autonomous LLM Ag…

SECTION 03 / 05

The **Metamodel**

How an agent armed with skills and rules turns business context into deployable artefacts, every step traceable.

03

Skills & rules wrap the whole lifecycle, from discovery to operations.

Far beyond DevOps: eight phases from business discovery to operations, each owned by a prefix, all tracing back to one metamodel.

DISCOVERY 3

Idea · Research · Workshop

SPECIFICATION 6

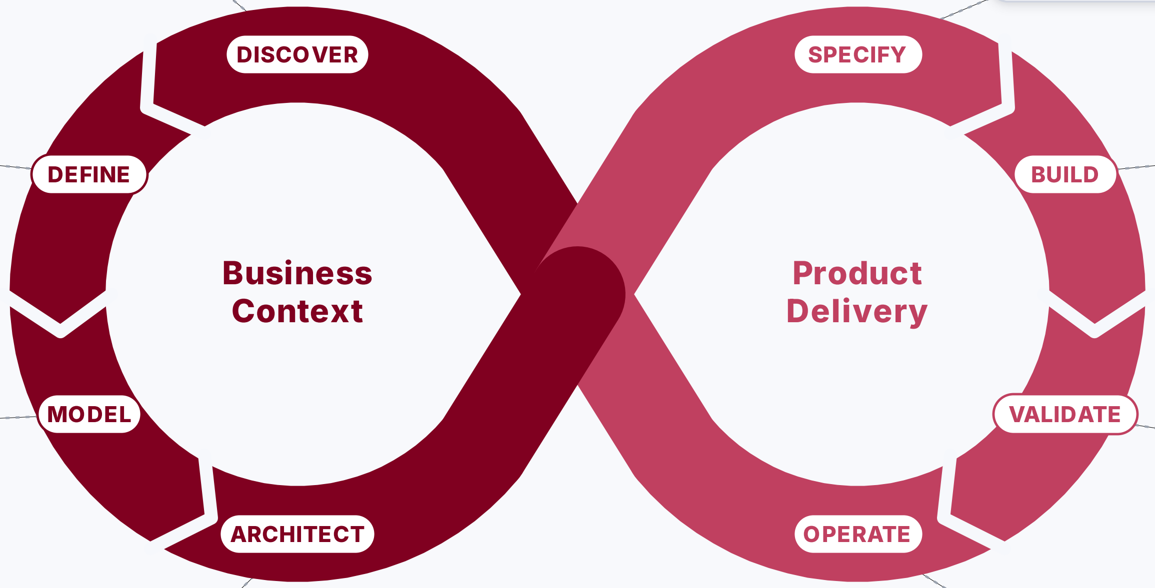
Functional breakdown · Use case · Quality attributes · PRD · Peer review · Grill-me

BUSINESS 9

Vision · Persona · Capability map · Value stream · Objective · Process · Model canvas · Quant model · Competitive landscape

DEVELOPMENT 8

Ralph loop · Git commit · Git worktree · PR · Git init · Agent config · Getting started · Stack guide



DOMAIN 3

Bounded context · Glossary · Domain model

QA 5 · planned

Test strategy · Test scenario · Test plan · Acceptance test · Eval harness

ARCHITECTURE 9

ADR · C4 · arc42 · UML · Structurizr · PlantUML · Service contract · CLI contract · Research

OPS 3

Runbook · Bug RCA · Terraform (Exoscale)

Discover & Define · make the context real.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Brainstorm & capture** — ideas, hypotheses
- **Research & scan** — market, competitors
- **Interview & workshop** — with the business
- **Model the business** — vision → objectives
- **Define the domain** — ubiquitous language

Business
9 SKILLS

- **Vision** — north star
- **Persona** — who we serve
- **Model canvas** — value model
- **Capability map** — what it can do
- **Competitive landscape** — market position
- **Value stream** — how value flows
- **Objective** — goals & KRs
- **Process** — operations
- **Quant model** — unit economics

Discovery
3 SKILLS

- **Idea** — capture & graduate
- **Research** — 1:1 interviews
- **Workshop** — group facilitation

Domain
3 SKILLS

- **Bounded context** — boundaries
- **Glossary** — ubiquitous language
- **Domain model** — aggregates & events

feeds → Design

Architect the solution · turn context into a solution.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Decide architecture**
  ADRs, C4, arc42
- **Define contracts**
  service, CLI
- **Break down function**
  FBS
- **Specify & qualify**
  use cases, quality
- **Write & review PRDs**
  one per epic

Architecture
9 SKILLS

- **ADR** — decisions
- **Research** — arch notes
- **Service contract** — API surface
- **CLI contract** — command surface
- **Structurizr** — C4 workspace
- **C4** — context → component
- **PlantUML** — UML pipeline
- **UML** — sequence/class/state
- **arc42** — arch narrative

Specification
6 SKILLS

- **Functional breakdown** — feature tree
- **Use case** — actor scenarios
- **Quality attributes** — ISO 25010
- **PRD** — one per epic
- **Peer review** — PRD/plan review
- **Grill-me** — adversarial review

UX
1 SKILL

- **Design system** — tokens & brand

Execute & Deliver · plan, build, test, ship.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Sequence the roadmap** — epics, phases
- **Plan increments** — atomic steps
- **Build & validate** — tests, checks
- **Deploy** — infra, release
- **Operate & fix** — runbooks, RCA

Specification
2 SKILLS

- **Delivery roadmap** — epics & phases
- **Implementation plan** — atomic increments

QA
5 · PLANNED

- **Test strategy** — pyramid & coverage
- **Test scenario** — from use cases
- **Test plan** — per release
- **Acceptance test** — Given–When–Then
- **Eval harness** — agent evals

Development
8 SKILLS

- **Agent config** — CLAUDE.md & rules
- **Getting started** — onboarding
- **Stack guide** — tech patterns
- **Git init** — repo setup
- **Git commit** — commits
- **Git worktree** — worktrees
- **PR** — pull requests
- **Ralph loop** — autonomous loop

Ops
3 SKILLS

- **Runbook** — operations
- **Bug RCA** — incident analysis
- **Terraform** — Exoscale infra

fed by ← Design

SECTION 04 / 05

The Build **Workflow**

From a PRD to merged code: how one engineer drives agents through alignment and verification gates, then lets them run autonomously.

04

From PRD to merged code: two fresh-session gates, then autonomy.

Business context and discovery are done; the roadmap gives one PRD per epic. The loop below runs per PRD.

1 Spec ready

ALREADY DONE

- Delivery roadmap → **one PRD per epic**
- Each PRD states what must be true

PRD in hand

2 Align & plan

FRESH SESSION

- `agent-grill-me`: agent grills you ~10 Q on impl & architecture
- you answer → shared mental model
- agent writes the **implementation plan**

Plan drafted

3 Verify the plan

FRESH SESSION

- `agent-peer-review`: findings ranked critical → low
- you resolve the ones needing guidance → amend
- `ralph audit`: plan is autonomously runnable

Audit green

4 Execute & ship

SEPARATE CLI

- `agent-ralph-loop ralph.sh`: fresh agent per increment
- implement → test gate → commit, until done
- run CI on the finished work

Merged

Two **fresh-session** checks (grill-me before, peer-review after) keep me and the agent aligned; the loop only runs once the plan is proven autonomous.

PRD says what must be true; the plan says how, safely.

Two artefacts, one chain: the PRD scopes the epic, the implementation plan breaks it into runnable increments.

BUILD BY FEATURE · ONE PER EPIC

PRD

"What must be true, and why."

- **Problem, goals & non-goals**: the scope boundary
- **User stories** grounded in personas (P-NN)
- **Acceptance criteria** + success metrics
- **§0 traceability**: the epic and its dependencies

E-NN · P-NN · C-N.M.FXX · QA-XXNN · UC-NN

1 PRD
↓
1 plan

→

acceptance criteria
become
the plan's test gates

increments
deliver
the PRD scope

ATOMIC INCREMENTS · ONE PER PRD

Implementation plan

"How we get there, reversibly."

- **Ordered increments** (Inc-1 … Inc-N)
- each **small** · **testable** · **reversible**
- scope + primary files per increment
- **test gate** + exit criteria = done

Plan-NNNN · Inc-N · Status: pending → done

Two fresh agents pressure-test the work, before and after the plan.

Grill-me pulls the thinking out of my head; peer-review checks the document. Both run in a clean session.

ALIGNMENT · BEFORE THE PLAN

`agent-grill-me`

Live Socratic session on the PRD, one question at a time.

- Agent asks ~10 focused Q on **implementation & architecture**
- Vocabulary checked against the **domain glossary**
- Decisions crystallised into **ADRs** while context is live
- Surfaces the **thinking gaps** only questioning reveals

Output: **shared mental model** → the agent then writes the implementation plan.

VERIFICATION · AFTER THE PLAN

`agent-peer-review`

Static, independent scan of the finished plan.

- Reads the plan end-to-end, builds a requirement map
- Findings ranked with concrete remediation:

  `critical` `major` `normal` `low`

- I resolve the ones needing guidance → **amend the plan**

Output: **validated plan** → ready for the autonomy audit.

Peer-review finds the **mechanical** gaps automatically; grill-me surfaces the **thinking** gaps under questioning.

The Ralph loop: a fresh agent per increment, until the plan is done.

WHAT THE LOOP IS

- A bash script (**ralph.sh**) drives the plan to completion, one increment per iteration.
- **Fresh agent each iteration** → clean context, no drift or context rot.
- Loops while increments are **pending**; stops when every one is **Status: done**.

```
ralph.sh <workspace> \
  --max-iterations 50 --with-prd --with-push
# spawns: claude --print < iteration-prompt
```

ONE ITERATION · PERCEIVE → IMPLEMENT → VALIDATE → COMMIT

Audit must be green before the first run.

Pick next pending increment
read workspace + plan

Implement its scope
follow the primary-files list

Run the test gate
pottery wheel, up to 3 retries

Commit the increment
advance status → done

repeat with a fresh agent

All increments done → run CI

BEFORE YOU GO

Key takeaways

Ten commandments for the developer, ten for the company. What to actually do on Monday.

Ten commandments for the developer.

The agent is a power tool. The judgement, the architecture and the guardrails are still yours.

1. **Build your own harness, but share what works.**
   No one tells you which IDE or agent to pick; pass the patterns and best practices back to the team.
2. **Treat your agent like a smart junior.**
   Give it tools, steer it, correct it.
3. **Keep a fresh context.**
   Start clean; don't let the window rot.
4. **Agents magnify bad architecture as fast as good.**
   So build the guardrails.
5. **The tooling is hackable.**
   It was built to be bent; adapt it to your workflow.
6. **Start minimal; earn every tool and framework.**
   Hit a wall before you add a tool; run the agent loop bare before you adopt an agentic framework.
7. **Fast to write is not a reason to write.**
   Coding was never the bottleneck; thinking was.
8. **Trust, but verify.**
   Read the diff; every line is yours to defend.
9. **Learn by playing.**
   Build hobby projects, not experiments on company goals.
10. **Refactoring got cheap. Architecture didn't get optional.**
    Cheap to change is not the same as fine to wing it.

⚓ Engineer the harness, not just the prompt. The craft moved **up a rung**; it didn't disappear.

Ten commandments for the company.

Building with agents is an organisational capability, not a gadget. Set the conditions.

1. **Enable the tooling.**
   Give every dev agents, sandboxes and access; don't gatekeep the lever.

2. **Standardize the workflow and artefacts, not the tool.**
   Mandate the method; let people pick their own harness.

3. **AI skips none of your real problems.**
   Quality, process, planning, architecture are still there; you just hit the wall faster, and it's bigger.

4. **Claim AI sovereignty now.**
   Don't turn every model and tool into a US dependency.

5. **Default to open source.**
   OSS always ends up the default; don't repeat the Microsoft Office lock-in.

6. **Govern the data before the agent sees it.**
   Decide what company data an agent may ingest; first, not after.

7. **Budget for maintenance.**
   The harness is a living system, not a one-off.

8. **Pay down cognitive debt.**
   Don't let agents quietly erode the team's understanding.

9. **Measure what must be true, not velocity.**
   Track traceability and correctness, not tickets closed.

10. **The method is the moat, not the model.**
    Invest in the harness and the memory; models are commodities.

⚓ The model is a commodity. Your edge is the **method and memory** you build around it.

THE TAKEAWAY

Engineer the harness, not just the prompt.

The model is the commodity. Your edge is the method, the memory and the guardrails you build around it.

Same model, better harness

The method is the moat

Standardize the method, swap the parts

Software engineering isn't dead. We just climbed another rung on the ladder of abstraction.